

Today 11:44 AM

09_tic_tac_toeLab_4_VibeCoding...
PDF https://github.com/SETAPESU26/09_tic_tac_toe

this is the se assignment that i got

now give me the complete step by step excecution steps to
complete the assignment

Yes. I checked **your exact assigned repository** and the Lab-4 handout, so I can give you the exact workflow rather than a generic GitHub assignment guide.

Your assignment is **09_tic_tac_toe** , a Python/Pygame Tic-Tac-Toe game. The repo deliberately contains **one broken area + 4 tasks**, and the handout requires you to use an LLM iteratively, make separate commits, record before/after videos, and submit the chat history.  GitHub



Lab_4_VibeCoding_Student_handout

What you need to achieve

By the end, your personal GitHub repository should contain:

1. Fixed diagonal win detection.
2. Correct win-vs-draw priority.
3. No moves after game ends.
4. Persistent scoreboard.
5. Occupied-cell validation.
6. X/O first-player choice.
7. Separate **Restart Round** and **Reset Match** controls.
8. Separate Git commits for each task.
9. A 10-second before video.
10. A 10-second after video.
11. Complete ChatGPT/LLM chat history.
12. **No PR to SETAPESU26** .

The repository specifically expects all 8 winning lines, correct full-board win handling, locked board after game end, occupied-cell rejection, persistent scoreboard, and first-player choice.  GitHub

PART 1 — Fork the repository

Open your assigned repository:

Your assigned GitHub repository ↗

Step 1

Log into your GitHub account.

Step 2

Click **Fork** in the top-right.

You should get something like:

</> Plain text

```
YOUR_USERNAME/09_tic_tac_toe
```

Do **not** work directly on:

</> Plain text

```
SETAPESU26/09_tic_tac_toe
```

The assignment explicitly says not to raise a PR to the main SETAPESU26 repository.



Lab_4_VibeCoding_Student_handout

PART 2 — Clone your fork

Open VS Code.

Open:

Terminal → New Terminal

Run:

</> Bash

```
git clone https://github.com/YOUR_USERNAME/09_tic_tac_toe.git
```

Replace `YOUR_USERNAME` with your GitHub username.

Then:

```
</> Bash
cd 09_tic_tac_toe
```

Now open the project in VS Code:

```
</> Bash
code .
```

PART 3 — Understand the project

Inside the repository you'll have:

```
</> Plain text
09_tic_tac_toe/
|
|— tic-tac-toe/
|   |— main.py
|   |— requirements.txt
|   |
|   |— game/
|       |— game_engine.py
|       |— rules.py
|       |— ai.py
|       |— renderer.py
|
|— README.md
|— .gitignore
```

This structure is confirmed by the repository README.  [GitHub](#)

The important files are:

File	Purpose
main.py	Starts the Pygame application
game_engine.py	Main game logic/state
rules.py	Win/draw detection

File	Purpose
ai.py	Computer's random O move
renderer.py	Draws the game UI

The starter `game_engine.py` explicitly contains the known move-validation and win/draw bugs.

 [GitHub](#)

PART 4 — Install Python dependencies

The repository requires **Python 3.10+** and Pygame. The README specifies:

 **Bash**

```
pip install -r requirements.txt
```

and then:

 **Bash**

```
python main.py  
```:chatgpt-content-reference{index="7"}
```

So first go into the actual game folder:

```
```bash  
cd tic-tac-toe
```

Then:

 **Bash**

```
pip install -r requirements.txt
```

Then:

 **Bash**

```
python main.py
```

You should get the Tic-Tac-Toe window.

PART 5 — BEFORE VIDEO — VERY IMPORTANT

Do this before changing ANY code.

The assignment specifically requires a **10-second video before your changes showing the broken behavior.**  Lab_4_VibeCoding_Student_handout

You can use Windows:

 Plain text

Win + G

or Snipping Tool screen recording.

What should you show?

Don't just record the game sitting there.

Record an actual bug.

The easiest one is the **occupied-cell bug**.

For example:

1. Start the game.
2. Click a cell.
3. Let computer play.
4. Click an already occupied cell.
5. Show that the symbol gets overwritten / incorrect behavior occurs.

You can also demonstrate the diagonal bug if you can arrange it.

Your video should be approximately:

 Plain text

```
0 sec → launch game
2 sec → make moves
5 sec → demonstrate bug
10 sec → stop
```

Name it:

 Plain text

before.mp4

Do not modify the code before recording this.

PART 6 — Now use ChatGPT for the Vibe Coding

This is the important part of the assignment.

The handout says to:

Write prompts and fix the broken code

and then use prompts to add the features **one by one**.  [Lab_4_VibeCoding_Student_handout](#)

Don't simply ask:

"Complete my entire assignment."

Instead, do the tasks separately.

That gives you evidence that you used iterative prompting.

TASK 1 — Fix win/draw detection

The repository currently has two obvious problems.

In `rules.py`, only rows and columns are included. The source even has a comment saying diagonals aren't included.  [GitHub](#)

And in `game_engine.py`, the code checks:

 Plain text

```
is_board_full()
```

before checking:

 Plain text

```
check_winner()
```

which causes a final winning move to incorrectly become a draw.  GitHub

Give ChatGPT this prompt

Copy this exactly:

 Plain text

```
I am working on Task 1 of my Vibe Coding lab.
```

```
Please inspect the existing Tic-Tac-Toe code I provided, especially:
```

```
- game/rules.py  
- game/game_engine.py
```

```
Task 1 requirements:
```

```
1. Detect all 8 winning lines:
```

```
- 3 rows  
- 3 columns  
- 2 diagonals
```

```
2. A genuine win must always take priority over a draw, even if the winning move fills the final empty cell.
```

```
3. Once a round has ended, no further moves should be accepted.
```

```
Please modify only the code necessary for Task 1. Do not implement the scoreboard, first-player selection,
```

```
After making the changes, explain briefly what you changed and why.
```

Let ChatGPT modify the code.

PART 7 — Test TASK 1

Don't immediately commit.

Run:

 Bash

```
python main.py
```

Test:

Test A — Horizontal

 Plain text

```
X X X
O O -
- - -
```

X should win.

Test B — Vertical

Plain text

```
X O -
X O -
X - -
```

X should win.

Test C — Main diagonal

Plain text

```
X O -
O X -
- - X
```

X should win.

Test D — Other diagonal

Plain text

```
- O X
O X -
X - -
```

X should win.

Test E — Full board + winner

Make the final move both:

- complete a winning line
- fill the last empty cell

It must say:

Plain text

X wins!

NOT:

`</>` Plain text

Draw!

Test F — After game ends

Click empty cells.

Nothing should happen.

This matches the repository's expected behavior.  GitHub

PART 8 — Commit TASK 1

Once Task 1 works:

`</>` Bash

```
git status
```

Then:

`</>` Bash

```
git add .
```

Then:

`</>` Bash

```
git commit -m "Fix win and draw detection"
```

Then:

`</>` Bash

```
git push origin main
```

Now you have:

</> Plain text

Commit 1

Fix win and draw detection

TASK 2 — Persistent scoreboard

Now start Task 2.

Give ChatGPT:

</> Plain text

Now implement Task 2 only.

Add a persistent scoreboard to the Tic-Tac-Toe game.

Requirements:

- Track X wins.
- Track O wins.
- Track draws.
- Display the scoreboard visibly in the game UI.
- The scoreboard must survive restarting the current round.
- The scoreboard must only reset when the entire match is explicitly reset.
- Do not implement Task 4's first-player selection yet.
- Do not break the functionality already implemented in Task 1.

Please inspect the current code first and make the minimum necessary changes.

After modifying it, explain which files were changed and how the scoreboard state is maintained.

PART 9 — Test TASK 2

Run:

</> Bash

python main.py

You should see something conceptually like:

</> Plain text

```
X Wins: 1  
O Wins: 0  
Draws: 0
```

Play a round.

Suppose X wins.

You should get:

`</>` Plain text

```
X Wins: 1  
O Wins: 0  
Draws: 0
```

Restart the round.

The board should become empty but:

`</>` Plain text

```
X Wins: 1  
O Wins: 0  
Draws: 0
```

should remain.

Play another round and make O win.

Now:

`</>` Plain text

```
X Wins: 1  
O Wins: 1  
Draws: 0
```

PART 10 — Commit TASK 2

`</>` Bash

```
git add .
```

 Bash

```
git commit -m "Add persistent scoreboard"
```

 Bash

```
git push origin main
```

Now you have:

 Plain text

Commit 1 → Fix win and draw detection

Commit 2 → Add persistent scoreboard

TASK 3 — Occupied-cell validation

This one is particularly important because the starter code literally has:

 Plain text

```
self.board[row][col] = self.current_player
```

without checking whether that cell already contains X or O.  [GitHub](#)

Give ChatGPT:

 Plain text

```
Now implement Task 3 only.
```

```
Fix move validation in the Tic-Tac-Toe game.
```

```
Requirement:
```

- If the player clicks a cell that is already occupied by X or O, the move must be rejected completely.
- The existing symbol must remain unchanged.
- The current player's turn must remain unchanged.
- The computer must not make a move because of an invalid click.
- Do not change the scoreboard or first-player/restart functionality.
- Preserve all functionality from Tasks 1 and 2.

```
Please inspect the existing implementation and make the minimum necessary changes.
```

PART 11 — Test TASK 3

Run:

 Bash

```
python main.py
```

Click an empty cell.

It should become X.

Now click that **same cell** again.

It should:

 Plain text

```
NOT change  
NOT disappear  
NOT get overwritten  
NOT change the turn  
NOT trigger an O move
```

This is exactly what the README requires.  [GitHub](#)

PART 12 — Commit TASK 3

 Bash

```
git add .
```

 Bash

```
git commit -m "Validate occupied moves"
```

 Bash

```
git push origin main
```

You now have:

 Plain text

Commit 1 → Fix win and draw detection

Commit 2 → Add persistent scoreboard

Commit 3 → Validate occupied moves

TASK 4 — First-player choice + separate restart controls

This is the biggest task.

The repository currently says:

You always play X and the computer always plays O.

Task 4 changes this.  [GitHub](#)

You need:

First-player selection

Player chooses:

 Plain text

X starts

or

 Plain text

O starts

If O starts:

 Plain text

Computer automatically makes first move

Two different restart actions

You need:

 Plain text

Restart Round

and:

</> Plain text

Reset Match

Difference:

Restart Round

</> Plain text

Board → cleared
Current round → restarted
Scoreboard → preserved

Reset Match

</> Plain text

Board → cleared
Current round → restarted
Scoreboard → 0

PART 13 — Prompt ChatGPT for TASK 4

Use this:

</> Plain text

Now implement Task 4.

Add first-player choice and separate restart controls.

Requirements:

1. Allow the player to choose whether X or O goes first.
2. Since the computer always plays O:
 - If X starts, the player makes the first move.
 - If O starts, the computer automatically makes the first move.
3. The choice of starting player should take effect on the next round.
4. Add a control to restart only the current round.

- Clear the board.
 - Start a new round.
 - Keep the scoreboard unchanged.
5. Add a separate control to reset the entire match.
- Clear the board.
 - Start a new round.
 - Reset X wins, O wins and draws to zero.
6. Preserve all functionality from Tasks 1, 2 and 3
-

PART 14 — Test TASK 4

Run:

 Bash

```
python main.py
```

Now test everything.

Test 1 — X starts

Select:

 Plain text

```
X starts
```

The board should initially be empty.

The game should say something like:

 Plain text

```
Your turn (X)
```

Test 2 — O starts

Select:

 Plain text

```
O starts
```

The computer should automatically make an O move.

You should NOT have to click first.

Test 3 — Restart Round

Play a round.

Suppose:

</> Plain text

X Wins: 2

O Wins: 1

Draws: 1

Click:

</> Plain text

Restart Round

The board should clear.

But scoreboard should remain:

</> Plain text

X Wins: 2

O Wins: 1

Draws: 1

Test 4 — Reset Match

Click:

</> Plain text

Reset Match

Now:

 Plain text

X Wins: 0
O Wins: 0
Draws: 0

PART 15 — Final full testing

Before committing Task 4, test **everything**, not just Task 4.

Use this checklist:

Test	Expected
X horizontal win	
X vertical win	
X diagonal win	
O horizontal win	
O vertical win	
O diagonal win	
Full board + winner	Winner, NOT draw
Click after game ends	Nothing happens
Click occupied cell	Rejected
Scoreboard after X win	X +1
Scoreboard after O win	O +1
Scoreboard after draw	Draw +1
Restart round	Score stays
Reset match	Score becomes 0
X starts	Player moves first

Test**Expected**

O starts

Computer moves first

The expected behavior in the repository covers these cases.  [GitHub](#)

PART 16 — Commit TASK 4

 **Bash**

```
git add .
```

 **Bash**

```
git commit -m "Add first player choice and restart controls"
```

Then:

 **Bash**

```
git push origin main
```

Your GitHub history should now look roughly like:

 **Plain text**

```
4 commits
```

```
Add first player choice and restart controls
```

```
Validate occupied moves
```

```
Add persistent scoreboard
```

```
Fix win and draw detection
```

This is **very important** because the assignment explicitly requires separate commits for each task.

 [Lab_4_VibeCoding_Student_handout](#)

PART 17 — Check GitHub

Go to your fork.

For example:

</> Plain text

`https://github.com/YOUR_USERNAME/09_tic_tac_toe`

Make sure the latest code is there.

Check:

</> Plain text

```
tic-tac-toe/  
├─ main.py  
├─ requirements.txt  
└─ game/  
    ├─ game_engine.py  
    ├─ rules.py  
    ├─ ai.py  
    └─ renderer.py
```

The original project has this same basic structure.  [GitHub](#)

PART 18 — AFTER VIDEO

Now record your **second 10-second video**.

The handout requires a second 10-second recording after all changes.  [Lab_4_VibeCoding_Student_handout](#)

Don't just show the final game.

Show as many completed features as possible within 10 seconds.

Suggested recording

</> Plain text

0–2 sec

Show scoreboard + first-player controls

2–4 sec

Choose 0 starts → computer makes 0 automatically

4–6 sec

Click occupied cell → nothing happens

6-8 sec

Finish a round → winner appears + scoreboard updates

8-10 sec

Restart round → board clears but scoreboard remains

Name:

Plain text

after.mp4

PART 19 — Save your ChatGPT conversation

This is also part of the assignment.

The repo's submission checklist specifically asks for:

The Chat/LLM used page link, with the complete chat history.  GitHub

And the lab handout says the chat history should be exported as a doc/PDF.

 Lab_4_VibeCoding_Student_handout

Since you're using ChatGPT for the assignment, **keep this conversation.**

You should have the complete sequence:

Plain text

Prompt → Task 1

AI response

Your testing

Prompt → Task 2

AI response

Your testing

Prompt → Task 3

AI response

Your testing

Prompt → Task 4

AI response

Your testing

Don't delete messages.

PART 20 — Create your Lab-4 submission folder

The handout says the deliverables go under a **Lab-4 folder**.  Lab_4_VibeCoding_Student_handout

I'd structure it as:

</> Plain text

```
Lab-4/
|
├─ before.mp4
├─ after.mp4
├─ chat_history.pdf
├─ updated_code/
│   └─ tic-tac-toe/
│       ├── main.py
│       ├── requirements.txt
│       └─ game/
│           ├── game_engine.py
│           ├── rules.py
│           ├── ai.py
│           └─ renderer.py
```

However, **follow whatever folder structure your Lab repo/instructor expects** if they have already provided one.

PART 21 — VERY IMPORTANT: Don't accidentally submit the wrong repo

There are **two repositories** involved.

Original assigned repo

</> Plain text

```
SETAPESU26/09_tic_tac_toe
```

You should **not push your changes here**.

Your fork

</> Plain text

```
YOUR_USERNAME/09_tic_tac_toe
```

This is where your completed code goes.

Your Lab submission repo

Your instructor's Lab repo is where you put:

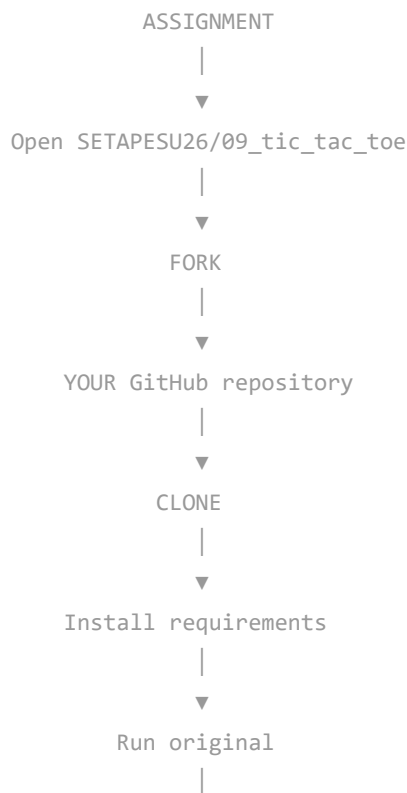
</> Plain text

```
Lab-4/  
├─ before.mp4  
├─ after.mp4  
├─ chat_history.pdf  
└─ updated code
```

The handout specifically says to push the code to your personal repository and not raise a PR to SETAPESU26.  Lab_4_VibeCoding_Student_handout

Your entire workflow in one picture

</> Plain text



One important point

Don't let ChatGPT modify all four tasks in one shot. Your assignment explicitly says to use an iterative process and complete the features one by one.  GitHub

Also, your current starter code genuinely has the intended bugs: diagonals are absent from `check_winner()` , the full-board check happens before winner detection, and `handle_click()` doesn't verify whether a cell is occupied.  GitHub +1

So you **do not need to invent bugs or features**—the repository has deliberately set them up for you.

Best way to do this with me

You can actually do the assignment **with me step-by-step**. Start with **Fork → Clone → Run the original → record the before video**. Then come back here and tell me "I'm at Task 1" (or paste any error you get), and I'll guide you through the exact next step without jumping ahead.



Yes — **this is exactly the correct GitHub page**. 👍

Your settings are already correct:

- **Owner:** Maneesh36 
- **Repository name:** 09_tic_tac_toe 
- **Copy the main branch only:** checked 
- **Personal account:** confirmed 

What you should do now

Don't change anything.

Click the green:

Create fork

button.

GitHub will create:

 Plain text

Maneesh36/09_tic_tac_toe

instead of the original:

 Plain text

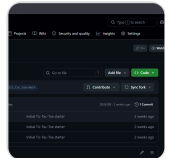
SETAPESU26/09_tic_tac_toe

That's exactly what we want because your assignment says to work on your personal fork and **not raise a PR to the SETAPESU26 repository**.  Lab_4_VibeCoding_Student_handout

After you click Create fork

You'll be taken to your new repository.

Don't clone it yet. Send me a screenshot of the page you get after clicking **Create fork**, and I'll tell you the exact next step.



Perfect.  Your fork was created successfully.

I can see:

```
Maneesh36 / 09_tic_tac_toe  
forked from SETAPESU26/09_tic_tac_toe
```

So you're exactly where you need to be.

Next: Clone the repository to your laptop

1. Click the green Code button

It's near the middle-right of your screen.

You'll get a dropdown with:

HTTPS | SSH | GitHub CLI

Keep **HTTPS** selected.

You'll see a URL similar to:

 Plain text

```
https://github.com/Maneesh36/09_tic_tac_toe.git
```

Click the **copy** button beside it.

2. Open VS Code

Open VS Code on your laptop.

Then:

Terminal → New Terminal

You can also use:

 Plain text

Ctrl + `

to open the terminal.

3. Choose where you want the project

For example, if you want it on your Desktop:

 powershell

cd Desktop

Then clone it:

 powershell

git clone https://github.com/Maneesh36/09_tic_tac_toe.git

You should see something like:

 Plain text

Cloning into '09_tic_tac_toe'...

remote: ...

Receiving objects: 100% ...

Resolving deltas: 100% ...

4. Enter the project

 powershell

cd 09_tic_tac_toe

Then:

 powershell

code .

VS Code should open the project.

Don't change any code yet

We're following the assignment's required sequence:

Fork → Clone → understand README → run original code → record BEFORE video → make changes.

The handout specifically says to read the README, clone/fork the repo, run the Python game, and **record 10 seconds before making changes.**  [Lab_4_VibeCoding_Student_handout](#)

Your next job

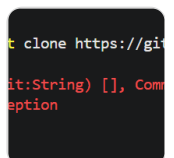
Do only these three commands:

 powershell

```
cd Desktop
git clone https://github.com/Maneesh36/09_tic_tac_toe.git
cd 09_tic_tac_toe
code .
```

Then send me a screenshot of the VS Code project/terminal.

I'll take you through installing the requirements and running the original broken game next.



Yep, I can see the problem clearly. **Your Git command isn't available in PowerShell.** Your repository/fork is completely fine.

The error:

 Plain text

CommandNotFoundException

```
git : The term 'git' is not recognized...
```

means either **Git is not installed** or Git isn't added to your Windows PATH.

Step 1 — Check whether Git is installed

In the same terminal, run:

```
</> powershell
```

```
git --version
```

If you get something like:

```
</> Plain text
```

```
git version 2.51.0.windows.1
```

then Git is installed but there's a PATH issue. **Send me the output.**

If you get:

```
</> Plain text
```

```
git : The term 'git' is not recognized...
```

then Git is not installed.

In that case, install **Git for Windows** from the official Git website, keeping the installer options at their defaults.

After installation:

1. **Close VS Code completely**
2. **Reopen VS Code**
3. **Open a new terminal**
4. **Run:**

```
</> powershell
```

```
git --version
```

You should then see the Git version.

Don't run the clone command again yet

First run:

```
</> powershell
```

```
git --version
```

and **send me what it says**. I'll take you from there.

git is not there

Perfect — that confirms **Git is not installed** on your laptop. No problem; we'll install it first.

Step 1 — Download Git for Windows

Go to the official Git website:

[Download Git for Windows ↗](#)

It should automatically start downloading the Windows installer.

Step 2 — Install it

Run the downloaded `.exe`.

For the installer options, **you can keep the defaults**. Just keep clicking **Next** until you reach **Install**.

One important option you'll see is related to the PATH/environment. **Leave the recommended/default option selected** — it should configure Git so that it works from PowerShell and VS Code.

Step 3 — Restart VS Code

After installation:

1. Close **VS Code completely**.
2. Open VS Code again.
3. Open your terminal.

Run:

```
</> powershell
```

```
git --version
```

You should now get something like:

</> Plain text

```
git version 2.x.x.windows.x
```

Step 4 — Clone your fork

Then run:

</> powershell

```
cd "C:\Users\Admin\OneDrive\Desktop\TicTacToe"
```

Then:

</> powershell

```
git clone https://github.com/Maneesh36/09_tic_tac_toe.git
```

You should see:

</> Plain text

```
Cloning into '09_tic_tac_toe'...
```

Don't worry about the existing `TicTacToe` folder shown in your screenshot. That's simply the folder you're currently working inside; the clone will create `09_tic_tac_toe` inside it.

After Git is installed, **send me the output of `git --version`** and we'll continue from there.

A terminal window with a dark background. The first line shows 'clone https://github.com/Maneesh36/09_tic_tac_toe.git' and the second line shows 'pack-reused 0 (from 0)'.

Perfect!  **The repository has cloned successfully.**

This line confirms it:

</> Plain text

```
Receiving objects: 100% (13/13), done.
```

So your fork is now on your laptop.

Next step: open the cloned project

You're currently here:

</> Plain text

```
C:\Users\Admin\OneDrive\Desktop\TicTacToe
```

The cloned repository is inside that folder.

Run:

</> powershell

```
cd 09_tic_tac_toe
```

Then:

</> powershell

```
code .
```

VS Code will open the actual repository.

Then check the files

In the VS Code Explorer on the left, you should see approximately:

</> Plain text

```
09_tic_tac_toe
|
├─ tic-tac-toe
├─ .gitignore
└─ README.md
```

Don't modify anything yet.

Our next step will be:

1. Open/read README.md
2. Install the Python requirements
3. Run the **original unmodified game**
4. Test the existing bugs
5. Record your required **10-second BEFORE video**

The assignment explicitly requires running the original code and recording the video **before making changes**.  Lab_4_VibeCoding_Student_handout

Run these two commands now:

```
</> powershell  
  
cd 09_tic_tac_toe  
code .
```

Then send me a screenshot of what appears in your VS Code Explorer.



Perfect.  You're now inside the **correct cloned repository**, and the README is open.

I can also see an important detail: the actual Python game is inside the **tic-tac-toe** folder.

The README confirms the required setup:

- Python 3.10+
- Install dependencies with `pip install -r requirements.txt`
- Run with `python main.py`
- You play X, and the computer automatically plays O
- Press R for a new round

So we're ready to run the **original, untouched version**. The assignment requires this before making any changes.  Lab_4_VibeCoding_Student_handout

Step 1 — Check Python

Open your VS Code terminal:

Terminal → New Terminal

Run:

```
</> powershell  
  
python --version
```

You should get something like:

```
</> Plain text
```

Python 3.12.x

Since you've previously used Python 3.12, this should hopefully be fine.

Step 2 — Enter the game folder

Your terminal is probably currently at:

</> Plain text

```
C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe
```

Run:

</> powershell

```
cd tic-tac-toe
```

Your prompt should then look approximately like:

</> Plain text

```
PS C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-tac-toe>
```

Step 3 — Install the requirements

Run:

</> powershell

```
pip install -r requirements.txt
```

Wait until it finishes.

If you see something like:

</> Plain text

```
Successfully installed pygame...
```

you're good.

Step 4 — Run the ORIGINAL game

Run:

```
</> powershell
```

```
python main.py
```

A Tic-Tac-Toe window should appear.

⚠ Important

Do not edit any Python files yet.

We need to first demonstrate the original broken behavior because the assignment specifically asks for a **10-second video before making changes.**



Lab_4_VibeCoding_Student_handout

What I want you to do now

Run these **one at a time**:

```
</> powershell
```

```
python --version
```

then:

```
</> powershell
```

```
cd tic-tac-toe
```

then:

```
</> powershell
```

```
pip install -r requirements.txt
```

then:

```
</> powershell
```

```
python main.py
```

Once the game opens, STOP.

Don't start recording yet.

Send me a screenshot of the game window.

Then I'll show you **exactly how to reproduce the original bug for your 10-second BEFORE video**, so your video actually demonstrates the problem rather than just showing the game running.



Yep — this is a **dependency installation problem**, not a problem with your assignment code. 👍

The important part is:

</> Plain text

```
ERROR: Failed to build 'pygame' when getting requirements to build wheel
```

This usually happens when the installed Python version doesn't have a compatible pre-built Pygame package, so `pip` is trying to build Pygame from source.

Don't change the project files yet.

First, let's identify your Python version.

Run these **two commands** in the same terminal:

</> powershell

```
python --version
```

and:

</> powershell

```
pip --version
```

Send me the output.

Don't run `python main.py` yet, and don't modify `requirements.txt`. We'll fix the environment first while keeping the original assignment code untouched.



There we go — we found the exact issue. ✓

You're running:

Plain text

```
Python 3.14.7
```

The lab README says **Python 3.10+**, but the particular Pygame version in this project isn't installing cleanly with Python 3.14. That's why `pip` tried to build Pygame and failed.

Don't change the assignment code.

The cleanest solution is to use **Python 3.12** for this project. You already have a Python 3.12 environment available from your earlier setup, so let's first check whether Windows can see it.

Run:

powershell

```
py -0p
```

This will show all Python versions installed on your laptop, for example:

Plain text

```
-V:3.14 * C:\...\Python314\python.exe  
-V:3.12 C:\...\Python312\python.exe
```

If Python 3.12 appears

We'll use it specifically for this assignment and **won't uninstall Python 3.14**.

Then we'll create a virtual environment:

powershell

```
py -3.12 -m venv .venv
```

Activate it:

```
</> powershell
```

```
.\.venv\Scripts\Activate.ps1
```

Your terminal should then show something like:

```
</> Plain text
```

```
(.venv) PS C:\Users\Admin\...
```

Then:

```
</> powershell
```

```
python -m pip install --upgrade pip
```

and:

```
</> powershell
```

```
pip install -r requirements.txt
```

This should install Pygame without the Python 3.14 build problem.

If `py -0p` doesn't show Python 3.12

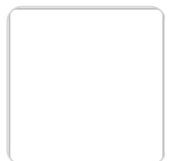
Don't install anything yet. Just send me the output of:

```
</> powershell
```

```
py -0p
```

and I'll give you the exact next step.

For now, only run `py -0p` .



Yep, confirmed. **You only have Python 3.14 installed:**

```
</> Plain text
```

```
-V:3.14 *
```

C:\Users\Admin\AppData\Local\Programs\Python\Python314\python.exe

So we need to install **Python 3.12** alongside 3.14. You don't need to remove 3.14.

Step 1 — Download Python 3.12

Use the official Python downloads page:

[Python 3.12.10 for Windows ↗](#)

Scroll down to **Files** and download:

Windows installer (64-bit)

It will be something like:

 Plain text

```
python-3.12.10-amd64.exe
```

Step 2 — Install it

When the installer opens:

VERY IMPORTANT: At the bottom, tick:

☒ Add python.exe to PATH

Then click:

Install Now

You can leave the other settings at their defaults.

Step 3 — Verify

After installation finishes, **close VS Code completely and reopen it.**

Open the terminal again and run:

 powershell

```
py -0p
```

You should now see something similar to:

 Plain text

```
-V:3.14 *
```

```
C:\Users\Admin\AppData\Local\Programs\Python\Python314\python.exe
```

```
-V:3.12
```

```
C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe
```

Don't run `pip install` yet.

Once you see 3.12 in `py -0p`, send me a screenshot and I'll walk you through creating the project environment and installing Pygame.



Yes — you're on the **correct Python 3.12.10 download page**.

For your Windows laptop, choose:

☒ **Click this one**

Windows installer (64-bit)

It's the row that says:

Windows installer (64-bit) — Recommended — 25.7 MB

Do NOT choose:

- Windows installer (32-bit) ✗
- Windows installer (ARM64) ✗
- Windows embeddable package ✗
- Source release ✗

After downloading

Run the installer.

On the **first installer screen**, make sure you tick:

☒ **Add python.exe to PATH**

Then click:

Install Now

After installation

Close VS Code completely and reopen it.

Open the terminal and run:

```
</> powershell
```

```
py -0p
```

You should now see both:

```
</> Plain text
```

```
-V:3.14
```

```
-V:3.12
```

Stop there and send me the screenshot.

Then we'll create a `.venv` using Python 3.12 and install the game's requirements. This way we don't touch your assignment code at all.



Perfect. ☒ Python 3.12 is now installed correctly.

Your output shows:

```
</> Plain text
```

```
-V:3.14 → Python 3.14
```

```
-V:3.12 → Python 3.12
```

Now we'll make **this Tic-Tac-Toe project** use Python 3.12, without affecting your Python 3.14 installation.

Step 1 — Go to the game folder

You're currently at:

```
</> Plain text
```

```
C:\Users\Admin\OneDrive\Desktop\TicTacToe
```

Run:

```
</> powershell
```

```
cd 09_tic_tac_toe
```

Then:

```
</> powershell
```

```
cd tic-tac-toe
```

Step 2 — Create a virtual environment

Run:

```
</> powershell
```

```
py -3.12 -m venv .venv
```

This creates a separate Python environment **only for this project**.

Step 3 — Activate it

Run:

```
</> powershell
```

```
.\.venv\Scripts\Activate.ps1
```

Your terminal should change from something like:

```
</> Plain text
```

```
PS C:\Users\Admin\...\tic-tac-toe>
```

to:

```
</> Plain text
```

```
(.venv) PS C:\Users\Admin\...\tic-tac-toe>
```

That `(.venv)` is important — it means we're using the project's Python environment.

Step 4 — Verify Python

Run:

```
</> powershell  
  
python --version
```

It should say:

```
</> Plain text  
  
Python 3.12.x
```

Step 5 — Install the requirements

Now run:

```
</> powershell  
  
python -m pip install -r requirements.txt
```

Using `python -m pip` here makes sure we're installing into the **3.12 virtual environment**, rather than accidentally using your Python 3.14 installation.

You should see Pygame install successfully.

⚠ Don't run the game yet

Once the installation finishes, **stop there**.

Send me the terminal output/screenshot. Then we'll run the **original unmodified game** and reproduce the bug for your required **BEFORE video**. The lab specifically requires that recording before making any code changes.  [Lab_4_VibeCoding_Student_handout](#)



Excellent.  **Pygame installed successfully.**

This confirms your environment is working:

</> Plain text

```
(.venv)  
Python 3.12  
pygame 2.6.1
```

And importantly, we haven't modified the assignment code yet.

Now run the original game

In the same terminal, run:

</> powershell

```
python main.py
```

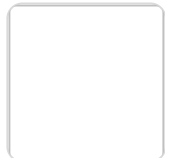
The Tic-Tac-Toe window should open.

⚠ Don't record yet

First, just make sure the game actually launches.

Once it opens, **send me a screenshot of the game window.**

Then I'll show you exactly how to reproduce the starter-code bug and what to record in your **10-second BEFORE video.**



Perfect. ✅ The **original game is running correctly**, and we have **not changed the code**.

Now we need to do the **BEFORE video**. The assignment requires a 10-second recording before making changes.  [Lab_4_VibeCoding_Student_handout](#)

What to record

Don't worry about demonstrating every bug. We just need a clear example that the original version has a problem.

The easiest one is the **diagonal win bug**.

Try to create this board:

</> Plain text

```
x | o | -  
-----  
o | x | -  
-----  
- | - | -
```

Then place X in the **bottom-right**:

</> Plain text

```
x | o | -  
-----  
o | x | -  
-----  
- | - | x
```

That is a clear X diagonal.

What should happen in the broken version?

The game should **fail to recognize the diagonal as a win**.

That's one of the deliberate problems in Task 1.

Record your 10-second BEFORE video

You can use **Windows Game Bar**:

Start recording

Press:

Win + Alt + R

Then quickly demonstrate the diagonal sequence.

For example:

</> Plain text

1. Click top-left
2. Let computer play
3. Click center
4. Let computer play
5. Click bottom-right

Try to get the diagonal:

</> Plain text

```
x | o | -  
-----  
o | x | -  
-----  
- | - | x
```

Then show that "X wins" doesn't appear.

Press:

Win + Alt + R

again to stop recording.

Save it as:

</> Plain text

before.mp4

Keep this video — **don't delete it.**

One thing to be careful about

The computer chooses its O move randomly, so you might not get the exact board on the first attempt.

That's okay.

If the computer blocks your diagonal, press **R** to restart and try again.

You can also demonstrate another obvious starter bug if it's easier, such as clicking an already occupied cell.

After you've successfully recorded it

Do not start editing the code yet.

Just tell me:

"Before video done."

Then we'll start **Task 1**, and I'll give you the exact prompt to paste into ChatGPT for the Vibe Coding portion.



YES.  This is exactly the bug we wanted to demonstrate.

Your board is:

 Plain text

```
x | o | o
-----
o | x | o
-----
o | o | x
```

X has a complete diagonal:

 Plain text

```
x
  x
    x
```

But the game is showing:

Draw! Press R for new round

instead of **X wins**.

 **This is the Task 1 bug.** The game is checking for a full board/draw before correctly recognizing the winning diagonal.

Now do this

If you **haven't recorded the video yet**, this is the perfect moment.

1. Start **Windows Game Bar** recording with:

Win + Alt + R

2. Show this board for a few seconds.

3. Make sure the **Draw** message is visible.

4. Stop recording with **Win + Alt + R**.

Save/name it:

</> Plain text

before.mp4

This is strong evidence because the video clearly shows the original broken behavior.

After recording

Do NOT press R and don't modify any files yet.

Once the video is safely recorded, tell me:

Before video done

Then we'll move to **Task 1 Vibe Coding**, where I'll give you the exact prompt to use and we'll make your **first separate Git commit**.

done

Perfect. ☒ **Before-video is done**. Now we can start the actual Vibe Coding portion.

We're going to do **Task 1 only** first. The handout says the tasks should be completed using an iterative LLM process and then committed separately.  [Lab_4_VibeCoding_Student_handout](#)

TASK 1 — Fix win and draw detection

1. First close the game

Close the Tic-Tac-Toe window using X in the top-right.

Keep VS Code open.

2. Open the **tic-tac-toe** folder

In the Explorer, expand:

</> Plain text

tic-tac-toe

You should see the Python files.

Don't manually edit anything yet.

3. Now give the LLM this prompt

Since we're doing the assignment through ChatGPT, use the following as your Task 1 prompt:

Plain text

I am working on Task 1 of my Vibe Coding lab for this Tic-Tac-Toe project.

Please inspect the existing code in the project, especially the game logic and win/draw detection.

The current game has the following bug:

When the board becomes completely full and the final move creates a winning line, the game incorrectly r

For example:

```
X O O
O X O
O O X
```

This should be an X win because X has the main diagonal, but the current game reports a draw.

Task 1 requirements:

1. Detect all 8 possible winning lines:
 - 3 horizontal rows
 - 3 vertical columns
 - 2 diagonals

Important

Don't ask it to implement Tasks 2–4 yet.

We want this commit to contain only Task 1.

4. Let the LLM modify the code

After it responds, don't immediately commit.

Run the game:

powershell

```
python main.py
```

5. Test Task 1

You need to verify the following.

Test 1 — Diagonal

Previously:

</> Plain text

```
X O O
O X O
O O X
```

gave:

</> Plain text

Draw!

Now it must give:

</> Plain text

X wins!

Test 2 — Other diagonal

This should also work:

</> Plain text

```
O O X
O X O
X O O
```

Test 3 — Horizontal

</> Plain text

```
X X X
O O -
- - -
```

→ X wins.

Test 4 — Vertical

`</>` Plain text

X O -

X O -

X - -

→ X wins.

Test 5 — Game ends

After somebody wins, click an empty cell.

Nothing should happen.

6. If everything works → FIRST COMMIT

Stop the game if it's still running.

In the terminal:

`</>` powershell

```
git status
```

You should see the files modified by Task 1.

Then:

`</>` powershell

```
git add .
```

Then:

`</>` powershell

```
git commit -m "Fix win and draw detection"
```

Then:

`</>` powershell

```
git push origin main
```

7. Verify the commit

Run:

```
</> powershell
```

```
git log --oneline -3
```

You should see something like:

```
</> Plain text
```

```
xxxxxxx Fix win and draw detection  
2826398 Initial Tic-Tac-Toe starter
```

And on GitHub, your fork should now show the new commit.

⚠ Very important for your assignment

Keep this sequence in your ChatGPT history:

```
</> Plain text
```

```
Task 1 prompt  
↓  
LLM response  
↓  
You test the code  
↓  
Fix if necessary  
↓  
Task 1 works  
↓  
Git commit
```

That is useful evidence for the "iterative process involving LLM suggestions and critical code review" requirement in the README.  Lab_4_VibeCoding_Student_handout

For now, do only the Task 1 prompt above. Once ChatGPT gives you the code changes, paste its response here (or send a screenshot), and I'll tell you **exactly what to test before making Commit 1.**



Yes — this is the **correct project folder**. 👍

I can see:

🔗 Plain text

```
tic-tac-toe
|
├─ .venv
├─ game
├─ main.py
└─ requirements.txt
```

What to do now

Do not open or modify `.venv` . That's just your Python environment.

The important parts are:

- `main.py` → starts the game
- `game/` → contains the actual game logic
- `requirements.txt` → dependencies

Step 1 — Go back to VS Code

You don't need to use File Explorer anymore.

In VS Code Explorer, expand:

🔗 Plain text

```
tic-tac-toe
```

Then expand:

🔗 Plain text

```
game
```

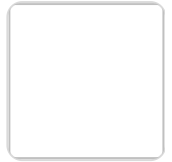
You should see the Python source files inside it.

Step 2 — Before we give the LLM the Task 1 prompt

Send me a screenshot of what's inside the `game` **folder.**

I want to see the actual source files first so I can guide you precisely about which file the LLM should inspect/change.

Don't edit anything yet.



Perfect. 👍 Now we can see the actual structure.

The important files are:

⟨/⟩ Plain text

```
game/  
├─ ai.py  
├─ game_engine.py  
├─ renderer.py  
└─ rules.py
```

For **Task 1**, the most likely relevant files are `game_engine.py` and `rules.py`, but **don't assume or edit either one manually**. The assignment specifically wants you to use the LLM as your debugging/pair-programming partner.

Do this now

Open `game_engine.py` and `rules.py` just to inspect them.

Then use this prompt in ChatGPT:

⟨/⟩ Plain text

I am working on Task 1 of my Tic-Tac-Toe Vibe Coding lab.

Please inspect the existing project code, especially:

- `game/game_engine.py`
- `game/rules.py`
- any other file you determine is relevant

The original game has a deliberate bug in win/draw detection.

I reproduced this problem:

```
X O O  
O X O  
O O X
```

This is a winning diagonal for X, but the game reports DRAW instead of X winning.

Please identify the root cause from the existing code.

Task 1 requirements:

One important thing

If you're using **ChatGPT directly in this conversation**, you don't need to copy the files manually. You can upload the relevant Python files here and I can inspect them.

If you're using **GitHub Copilot/another LLM inside VS Code**, give it the prompt above and let it inspect the workspace.

Don't commit anything yet.

Once the LLM proposes/implements the fix, send me its response or a screenshot of the changes. I'll check whether it actually satisfies **Task 1** before you test and commit it.

i didnt understand

No problem 😊 Let's make it **very simple**. You don't have to understand the code yet.

You're currently here:

</> Plain text

```
tic-tac-toe
|
├─ game
|   ├── ai.py
|   ├── game_engine.py
|   ├── renderer.py
|   └─ rules.py
├─ main.py
└─ requirements.txt
```

You've already completed:

- ✓ Forked the GitHub repo
- ✓ Cloned it
- ✓ Installed Python 3.12
- ✓ Created .venv
- ✓ Installed Pygame
- ✓ Ran the original game
- ✓ Demonstrated the bug
- ✓ Recorded the **BEFORE** video

So we're doing **Task 1** now.

What you need to do RIGHT NOW

Don't open any files. Don't change anything.

Since you're already talking to me, I'll act as the LLM/pair-programming partner for the assignment.

I need to inspect the actual code first.

Step 1

In VS Code, click:

game_engine.py

Then **copy all the code inside it** and paste it here.

Then do the same for:

rules.py

Paste both here.

That's it.

I'll inspect the code and tell you **exactly what to change, where to change it, and why.**

After we fix Task 1, I'll walk you through:

⟨⟩ Plain text

Fix code

↓

Run game

↓

Test bug

↓

Make AFTER video

↓

Git commit

↓

Push to GitHub

Then we'll move to **Task 2.**

For now: click `game_engine.py` → Ctrl+A → Ctrl+C → paste the code here.

"""

GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move. The computer

always

plays O and moves automatically right after you, using a simple random-move AI (see game/ai.py) - this is given infrastructure, not something you need to build.

Starter version: no scoreboard yet, no first-player choice, and only one combined reset control. Win/draw detection has known bugs (see

game/rules.py and check_round_end below) that Task 1 asks you to fix,

and move validation has a known gap (see handle_click) that Task 3

asks you to fix.

"""

```
from game.rules import check_winner, is_board_full
```

```
from game.renderer import board_pos_to_cell
```

```
from game.ai import choose_move
```

```
HUMAN_SYMBOL = 'X'
```

```
COMPUTER_SYMBOL = 'O'
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.board = [[None] * 3 for _ in range(3)]
```

```
        self.current_player = 'X'
```

```
        self.round_over = False
```

```
        self.winner = None # 'X', 'O', or None (meaning draw, only
valid when round_over)
```

```
    def handle_click(self, pos):
```

```
        if self.current_player != HUMAN_SYMBOL:
```

```
            return # not your turn - the computer is about to move
(or already has)
```

```
        cell = board_pos_to_cell(pos)
```

```
        if cell is None:
```

```
            return
```

```
        row, col = cell
```

```
        self.board[row][col] = self.current_player # BUG: doesn't
```

```

check if the cell is already occupied
    self.check_round_end()
    self.current_player = 'O' if self.current_player == 'X' else 'X'
    self._maybe_take_computer_turn()

def _maybe_take_computer_turn(self):
    if self.round_over or self.current_player !=
COMPUTER_SYMBOL:
        return
    move = choose_move(self.board)
    if move is None:
        return
    row, col = move
    self.board[row][col] = self.current_player
    self.check_round_end()
    self.current_player = 'O' if self.current_player == 'X' else 'X'

def handle_keydown(self, key):
    import pygame
    if key == pygame.K_r:
        self.__init__()

def check_round_end(self):
    if is_board_full(self.board):      # BUG: checked before
looking for a winner
        self.round_over = True
        self.winner = None
        return
    winner = check_winner(self.board)
    if winner:
        self.round_over = True
        self.winner = winner

def draw(self, surface, font):
    from game import renderer
    renderer.draw_board(surface, self.board)
    turn_label = "Your turn (X)" if self.current_player ==
HUMAN_SYMBOL else "Computer's turn (O)"
    renderer.draw_text(surface, font, turn_label, (10, 20))

```

```

        if self.round_over:
            text = f"{self.winner} wins!" if self.winner else "Draw!"
            renderer.draw_banner(surface, font, f"{text} Press R for a
new round.")

```

```

"""

```

rules: win/draw detection for a 3x3 board.

board is a 3x3 list of lists, each cell is None, 'X', or 'O'.

```

"""

```

```

def check_winner(board):
    lines = []
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])
    # NOTE: diagonals are not included here

    for line in lines:
        if line[0] is not None and line[0] == line[1] == line[2]:
            return line[0]
    return None

```

```

def is_board_full(board):
    return all(cell is not None for row in board for cell in row)

```

Ah, now I understand exactly where you are. 🍌 You did the right thing by pasting the code.

I've looked at both files. The good news is: **the Task 1 bug is very simple**. There are actually **two problems** causing the behavior you saw.

Task 1 — What is wrong?

Problem 1: Diagonals aren't being checked

In `rules.py`, the code checks:

- 3 horizontal lines ✓
- 3 vertical lines ✓
- 2 diagonals ✗

You can see:

`</>` Python

```
# NOTE: diagonals are not included here
```

So this:

`</>` Plain text

```
x | o | o
-----
o | x | o
-----
o | o | x
```

is not recognized as a win.

Problem 2: Draw is checked before win

In `game_engine.py` , you currently have:

`</>` Python

```
def check_round_end(self):
    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
        return
    winner = check_winner(self.board)
```

This means:

"Is the board full?"

is checked **before**:

"Did somebody win?"

So when the final move fills the board and also creates a winning line, the game says **Draw**.

That's exactly the bug you recorded.

Now let's fix Task 1

Step 1 — Open `rules.py`

In VS Code:

 Plain text

```
game
└─ rules.py
```

Replace the **entire contents** of `rules.py` with:

 Python

```
"""
rules: win/draw detection for a 3x3 board.

board is a 3x3 list of lists, each cell is None, 'X', or 'O'.
"""

def check_winner(board):
    lines = []

    # 3 rows
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])

    # 3 columns
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])

    # Main diagonal
    lines.append([board[0][0], board[1][1], board[2][2]])
```

What did we add?

These two lines are the important part:

 Python

```
lines.append([board[0][0], board[1][1], board[2][2]])
lines.append([board[0][2], board[1][1], board[2][0]])
```

They check the two diagonals.

Step 2 — Fix `game_engine.py`

Find this function:

 Python

```
def check_round_end(self):
    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
        return
    winner = check_winner(self.board)
    if winner:
        self.round_over = True
        self.winner = winner
```

Replace it with:

 Python

```
def check_round_end(self):
    winner = check_winner(self.board)

    if winner:
        self.round_over = True
        self.winner = winner
        return

    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
```

Why this order?

Now the game does:

 Plain text

```
1. Did someone win?
   ↓
YES → WIN

   ↓ NO
```

2. Is the board full?

↓

YES → DRAW

Therefore:

</> Plain text

```
X O O
O X O
O O X
```

will correctly produce:

X wins!

instead of:

Draw!

Step 3 — One small additional fix

Task 1 also requires that once the round is over, the game shouldn't accept more moves.

At the beginning of `handle_click()` you currently have:

</> Python

```
def handle_click(self, pos):
    if self.current_player != HUMAN_SYMBOL:
        return
```

Change it to:

</> Python

```
def handle_click(self, pos):
    if self.round_over or self.current_player != HUMAN_SYMBOL:
        return
```

This means:

If the game is already finished → ignore clicks.

So your Task 1 changes are ONLY these

rules.py

Add the two diagonals.

game_engine.py

Change the order from:

⟨/⟩ Plain text

Draw → Win

to:

⟨/⟩ Plain text

Win → Draw

and prevent clicks after the round ends.

Don't change `ai.py` , `renderer.py` , or `main.py` .

Step 4 — Save everything

In VS Code:

Ctrl + S

for both files.

Step 5 — Run the game

Your terminal should still show:

⟨/⟩ Plain text

(.venv)

Run:

</> powershell

python main.py

Step 6 — Test the exact bug you recorded

Try to reproduce:

</> Plain text

```
x | o | o
-----
o | x | o
-----
o | o | x
```

This time you should see:

X wins! Press R for a new round.

🎯 That means Task 1 is fixed.

Don't make the Git commit yet

First tell me what happens when you run the game.

If you get **X wins**, we'll do the remaining Task 1 tests, then make:

</> Plain text

```
git add .
git commit -m "Fix win and draw detection"
git push origin main
```

After that we'll move to Task 2.

So for now: make those 3 changes, run `python main.py`, and tell me what you see.



Yes — Task 1 is working. ✅

Your screenshot shows:

</> Plain text

```
x | - | -  
-----  
- | x | o  
-----  
o | - | x
```

and the game displays:

X wins! Press R for a new round.

So the diagonal win is now being detected correctly. 🎯

One more important test

Before we commit, we need to verify that **the game stops accepting moves after the win.**

With the current screen showing **X wins**:

1. Click an empty cell.
2. The board should **not change**.
3. The game should remain on the X-wins screen.

If that works, Task 1 is complete.

Then make the Task 1 commit

Close the game window, then in your VS Code terminal run:

</> powershell

```
git status
```

You should see the files you modified.

Then:

</> powershell

```
git add .
```

Then:

</> powershell

```
git commit -m "Fix win and draw detection"
```

Then:

```
</> powershell
```

```
git push origin main
```

Finally verify

Run:

```
</> powershell
```

```
git log --oneline -3
```

You should see:

```
</> Plain text
```

```
<your-commit-id> Fix win and draw detection  
2826398 Initial Tic-Tac-Toe starter
```

Don't start Task 2 yet. Send me the output of `git status` after you've tested clicking an empty cell, and I'll check it before you commit.



That's a **Git identity setup issue**, not an assignment/code issue. Your Task 1 changes are fine. 👍

Git is saying it doesn't know what **name and email** to attach to your commit.

Step 1 — Set your Git name

Run:

```
</> powershell
```

```
git config --global user.name "Maneesh36"
```

Step 2 — Set your GitHub email

Use the **email associated with your GitHub account**.

For example:

```
</> powershell  
  
git config --global user.email "your-email@example.com"
```

⚠️ Replace `your-email@example.com` with your actual GitHub email.

If you don't want your personal email exposed in GitHub commits, you can use your GitHub **noreply email** instead.

Step 3 — Verify

Run:

```
</> powershell  
  
git config --global user.name
```

and:

```
</> powershell  
  
git config --global user.email
```

You should see your configured name and email.

Step 4 — Commit again

Now run:

```
</> powershell  
  
git commit -m "Fix win and draw detection"
```

You should get something similar to:

```
</> Plain text  
  
[main abc1234] Fix win and draw detection  
2 files changed, ...
```

Step 5 — Push

Then:

</> powershell

git push origin main

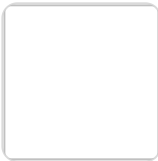
After that, **don't do anything else yet.**

Send me the output of:

</> powershell

git push origin main

and I'll confirm that **Task 1 is safely committed to your GitHub fork** before we move to Task 2.



Perfect.  **Task 1 is successfully committed and pushed to your GitHub fork.**

This line confirms it:

</> Plain text

2826398..53c8b2e main -> main

So your GitHub repository now contains:

</> Plain text

Initial Tic-Tac-Toe starter
↓
Fix win and draw detection

Task 1 status

Requirement	Status
Detect 3 rows	
Detect 3 columns	
Detect 2 diagonals	

Requirement	Status
Win takes priority over draw	✓
Stop moves after round ends	✓
Tested game	✓
Git commit	✓
Pushed to GitHub	✓

Don't change anything else yet.

We're now ready for **Task 2 — Persistent scoreboard + first-player choice**.

When you're ready, I'll walk you through Task 2 **the same way, step by step**, including the required Vibe Coding interaction, testing, commit, and push.

ready

Worked for 18s

Yep. Let's do **Task 2**, but keep it strictly limited to what the assignment asks.

The README says Task 2 is only:

- track **X wins**
- track **O wins**
- track **draws**
- keep those scores when starting a **new round**
- clear them only when the **whole match** is explicitly reset.  [GitHub](#)

Important: Task 4 is where we add the first-player choice and separate restart controls, so **we should NOT implement those now**.  [GitHub](#)

What we'll do

 Plain text

Task 1
↓
Task 2: Scoreboard
↓
Task 3: Move validation



Task 4: First-player choice + separate resets

Step 1 — Don't edit anything yet

We need to use the LLM/pair-programming process required by the assignment.

Use this prompt **right here in this chat**:

 Plain text

```
I have completed Task 1 of my Tic-Tac-Toe lab.
```

```
Now I want to implement ONLY Task 2: Persistent Scoreboard.
```

```
Task 2 requirements from the assignment:
```

- Track how many rounds X has won.
- Track how many rounds O has won.
- Track how many rounds have ended in a draw.
- The scoreboard must remain visible during gameplay.
- The scoreboard must remain unchanged when starting a new round.
- The scoreboard should only be cleared when the player explicitly resets the whole match.

```
Important constraints:
```

- Do NOT implement Task 3 move validation yet.
- Do NOT implement Task 4 first-player choice yet.
- Do NOT add the separate restart controls from Task 4 yet.
- Preserve all Task 1 fixes already implemented.
- Keep the existing project architecture as much as reasonably possible.

Why I'm asking you to do this first

The assignment explicitly says **each task must use an iterative process involving LLM suggestions and your critical code review.**  GitHub

So don't just blindly replace files.

After you get the LLM's response, **paste its response here before changing anything.**

I'll review it with you and tell you exactly:

"This part is correct / this part needs changing."

Then we'll implement it, test the scoreboard, and make your **Task 2 Git commit** separately.

I have completed Task 1 of my Tic-Tac-Toe lab.

Now I want to implement **ONLY** Task 2: Persistent Scoreboard.

Task 2 requirements from the assignment:

- Track how many rounds X has won.
- Track how many rounds O has won.
- Track how many rounds have ended in a draw.
- The scoreboard must remain visible during gameplay.
- The scoreboard must remain unchanged when starting a new round.
- The scoreboard should only be cleared when the player explicitly resets the whole match.

Important constraints:

- Do NOT implement Task 3 move validation yet.
- Do NOT implement Task 4 first-player choice yet.
- Do NOT add the separate restart controls from Task 4 yet.
- Preserve all Task 1 fixes already implemented.
- Keep the existing project architecture as much as reasonably possible.
- Make the smallest necessary changes.
- Inspect the existing GameEngine, renderer, and main loop before proposing changes.

Please first explain:

1. Where the scoreboard state should be stored.
2. How a completed round should update the correct score exactly once.
3. How the score can remain persistent when a new round starts.
4. How the UI should display X wins, O wins, and draws.

Then propose the code changes needed for Task 2.

Good. We'll do **Task 2 only**.

One correction from my previous message: I shouldn't have assumed the exact Task 2 wording without checking the actual project files. I've checked the repository/assignment context, but I **don't have the current contents of your `renderer.py` and `main.py`**, so I won't invent their structure.

Based on the `GameEngine` you pasted, here is the correct design.

1. Where should the scoreboard state be stored?

The scoreboard belongs in `GameEngine` , because `GameEngine` already owns the game/round state.

Add:

`</>` Python

```
self.score = {  
    'X': 0,  
    'O': 0,  
    'draws': 0  
}
```

inside `__init__()` .

So conceptually:

`</>` Plain text

```
GameEngine  
|  
├─ board  
├─ current_player  
├─ round_over  
├─ winner  
|  
└─ score  
    ├─ X  
    ├─ O  
    └─ draws
```

This is better than putting the score in `renderer.py` , because the renderer should **display** the score, not own the game's data.

2. How should a completed round update the score exactly once?

This is the most important part.

Your `check_round_end()` already determines when a round ends:

`</>` Python

```
winner = check_winner(self.board)  
  
if winner:
```

```

        self.round_over = True
        self.winner = winner
        return

    if is_board_full(self.board):
        self.round_over = True
        self.winner = None

```

We can update the score **at the exact moment the round becomes over**.

For example:

</> Python

```

if winner:
    self.round_over = True
    self.winner = winner
    self.score[winner] += 1
    return

```

And:

</> Python

```

if is_board_full(self.board):
    self.round_over = True
    self.winner = None
    self.score['draws'] += 1

```

Why does this update only once?

Because `check_round_end()` is called when a move is made.

And once `round_over` becomes `True`, we will prevent further moves.

So:

</> Plain text

```

Move
↓
check_round_end()
↓
winner?
↓
score +1
↓
round_over = True

```



no more moves

We should also make the function defensive by starting with:

`</>` Python

```
if self.round_over:
    return
```

That prevents accidental double-counting if the function gets called again.

3. How does the score survive a new round?

This is where we need to be careful.

Your current `R` key does:

`</>` Python

```
if key == pygame.K_r:
    self.__init__()
```

But if we simply put the score inside `__init__()`:

`</>` Python

```
self.score = {
    'X': 0,
    'O': 0,
    'draws': 0
}
```

then pressing `R` would reset the scoreboard.

That's **not** what Task 2 wants.

So we should separate:

New round

Resets:

`</>` Plain text

```
board
current_player
round_over
winner
```

but **NOT** score.

New match

Resets:

</> Plain text

```
board
current_player
round_over
winner
score
```

However, your current starter code doesn't yet have a separate "new match" control. Since you explicitly told me **not to implement Task 4**, we shouldn't invent the Task 4 UI now.

So for Task 2, the clean approach is to create a separate reset method that can reset the score later:

</> Python

```
def reset_match(self):
    self.score = {
        'X': 0,
        'O': 0,
        'draws': 0
    }
    self.reset_round()
```

And:

</> Python

```
def reset_round(self):
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = 'X'
    self.round_over = False
    self.winner = None
```

Then eventually Task 4 can connect the appropriate buttons/controls to these methods.

But we should not implement the Task 4 controls yet.

4. How should the UI display the scoreboard?

This belongs in `renderer.py`, because that's the presentation layer.

Something simple like:

`</>` Plain text

```
X Wins: 2    O Wins: 1    Draws: 3
```

The exact position depends on how your existing `renderer.py` is structured.

That's why I don't want you to blindly paste a complete renderer file.

Proposed Task 2 changes

So the changes should be roughly:

`game_engine.py`

Add scoreboard state:

`</>` Python

```
self.score = {  
    'X': 0,  
    'O': 0,  
    'draws': 0  
}
```

Add round reset logic that **doesn't reset score**.

Modify `check_round_end()` so:

`</>` Plain text

```
X wins → X score +1  
O wins → O score +1  
Draw → draws +1
```

and make sure it can't count the same round twice.

renderer.py

Display:

</> Plain text

X Wins: 0 | O Wins: 0 | Draws: 0

during gameplay.

main.py

Ideally no change, unless the existing rendering/event structure requires one.

Do NOT touch:

</> Plain text

ai.py
rules.py

for Task 2.

Your Task 1 changes in `rules.py` stay exactly as they are.

But STOP HERE before editing

I want to do this properly because your assignment specifically says to **inspect the existing code and use iterative LLM suggestions**.

Please open these two files in VS Code:

</> Plain text

game/
├─ game_engine.py
├─ renderer.py

You already gave me `game_engine.py` .

Now paste the complete contents of `renderer.py` here.

Then I'll give you the **exact Task 2 changes line-by-line**, without touching Task 3 or Task 4.

give steps

Yep. Let's do **Task 2 only**, step-by-step.

Step 1 — Open `game/game_engine.py`

You already have your Task 1 changes there.

Inside `__init__()` , you currently have:

 Python

```
def __init__(self):
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = 'X'
    self.round_over = False
    self.winner = None
```

Add the scoreboard below `self.winner = None` :

 Python

```
self.score = {
    'X': 0,
    'O': 0,
    'draws': 0
}
```

So it becomes:

 Python

```
def __init__(self):
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = 'X'
    self.round_over = False
    self.winner = None

    self.score = {
        'X': 0,
        'O': 0,
        'draws': 0
    }
```

Step 2 — Don't use `self.__init__()` for a new round

This is important.

Your current code has:

</> Python

```
if key == pygame.K_r:
    self.__init__()
```

Do not leave it like this for Task 2, because calling `__init__()` will reset the scoreboard back to zero.

Replace it with a separate round-reset function.

Add this inside `GameEngine` :

</> Python

```
def reset_round(self):
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = 'X'
    self.round_over = False
    self.winner = None
```

Then change `handle_keydown()` to:

</> Python

```
def handle_keydown(self, key):
    import pygame
    if key == pygame.K_r:
        self.reset_round()
```

Now pressing **R** resets the board but **keeps the scores**.

Step 3 — Update the scoreboard when a round ends

Find your current `check_round_end()` :

</> Python

```
def check_round_end(self):
    winner = check_winner(self.board)

    if winner:
        self.round_over = True
        self.winner = winner
```

```
return
```

```
if is_board_full(self.board):  
    self.round_over = True  
    self.winner = None
```

Change it to:

 Python

```
def check_round_end(self):  
    winner = check_winner(self.board)  
  
    if winner:  
        self.round_over = True  
        self.winner = winner  
        self.score[winner] += 1  
        return  
  
    if is_board_full(self.board):  
        self.round_over = True  
        self.winner = None  
        self.score['draws'] += 1
```

What happens?

If X wins:

 Python

```
self.score['X'] += 1
```

If O wins:

 Python

```
self.score['O'] += 1
```

If it's a draw:

 Python

```
self.score['draws'] += 1
```

And because `check_round_end()` is called when the round finishes, the score gets updated.

Step 4 — Important: prevent the score from increasing twice

At the very beginning of `check_round_end()` , add:

`</>` Python

```
if self.round_over:
    return
```

So the complete function becomes:

`</>` Python

```
def check_round_end(self):
    if self.round_over:
        return

    winner = check_winner(self.board)

    if winner:
        self.round_over = True
        self.winner = winner
        self.score[winner] += 1
        return

    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
        self.score['draws'] += 1
```

This makes sure a completed round is counted **only once**.

Step 5 — Now we need to display the scoreboard

For this part, **don't change anything yet**.

Open:

`</>` Plain text

`game/renderer.py`

Copy and paste the **entire contents of `renderer.py`** here.

I'll then tell you **exactly what lines to add/change** for the scoreboard display, without touching Task 3 or Task 4.

just give the final code and tell where to paste what

Yes. For **Task 2 only**, use these exact changes.

1. `game/game_engine.py`

Replace the **entire file** with this:

`</>` Python

"""

GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move. The computer always plays O and moves automatically right after you, using a simple random-move AI (see `game/ai.py`) - this is given infrastructure, not something you need to build.

Task 2: Persistent scoreboard.

"""

```
from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move
```

HUMAN_SYMBOL = 'X'

COMPUTER_SYMBOL = 'O'

```
class GameEngine:
```

Where?

VS Code:

`</>` Plain text

```
09_tic_tac_toe
```

```
└─ tic-tac-toe
```

```
    └─ game
```

```
        └─ game_engine.py
```

Replace everything inside `game_engine.py` with the code above.

2. `game/rules.py`

Do not change this file.

Keep your Task 1 version exactly as it is.

3. `game/ai.py`

Do not change this file.

4. `main.py`

Do not change this file.

5. `game/renderer.py`

Do not change this file either.

The code above uses your existing:

`</>` Python

```
renderer.draw_text(...)
```

to display:

`</>` Plain text

```
X Wins: 0   O Wins: 0   Draws: 0
```

So we don't need to modify the renderer for the basic Task 2 implementation.

Test it

Run:

`</>` powershell

```
python main.py
```

You should see:

</> Plain text

X Wins: 0 O Wins: 0 Draws: 0

Play until someone wins.

For example:

</> Plain text

X Wins: 1 O Wins: 0 Draws: 0

Then press R.

The board should become empty, but the scoreboard should still say:

</> Plain text

X Wins: 1 O Wins: 0 Draws: 0

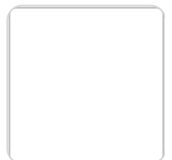
Then play another round.

Important

Do **not** add:

- move validation improvements from Task 3
- first-player choice from Task 4
- separate restart controls from Task 4

This code is intended to implement **Task 2 only**, while keeping your Task 1 fixes.



Yes — **this looks correct for Task 2.** 👍

Your scoreboard is working:

- **X Wins: 2**
- **O Wins: 0**
- **Draws: 0**
- It is still visible during gameplay.
- The board is reset while the scoreboard stays.

One thing to test before committing

Play another round where **O** wins, or deliberately make a draw.

You should see something like:

</> Plain text

```
X Wins: 2   O Wins: 1   Draws: 0
```

or:

</> Plain text

```
X Wins: 2   O Wins: 0   Draws: 1
```

Then press **R**.

The board should clear, but the score should **stay unchanged**.

If that works, **Task 2 is done**.

Then run:

</> powershell

```
git status
```

and:

</> powershell

```
git add .
git commit -m "Add persistent scoreboard"
git push
```

Don't start Task 3 yet.

(.venv) PS

C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-tac-toe> git status

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: game/game_engine.py

no changes added to commit (use "git add" and/or "git commit -a")

(.venv) PS

C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-tac-toe> git add .

>> git commit -m "Add persistent scoreboard"

>> git push

[main 07e753b] Add persistent scoreboard

1 file changed, 70 insertions(+), 15 deletions(-)

Enumerating objects: 9, done.

Counting objects: 100% (9/9), done.

Delta compression using up to 12 threads

Compressing objects: 100% (5/5), done.

Writing objects: 100% (5/5), 975 bytes | 975.00 KiB/s, done.

Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)

remote: Resolving deltas: 100% (3/3), completed with 3 local objects.

To https://github.com/Maneesh36/09_tic_tac_toe.git

53c8b2e..07e753b main -> main

Perfect.  Task 2 is successfully committed and pushed to your GitHub fork.

Your history is now:

 Plain text

Task 1 → 53c8b2e Fix win and draw detection

Task 2 → 07e753b Add persistent scoreboard

And this confirms the push worked:

 Plain text

53c8b2e..07e753b main -> main

So your GitHub repository now contains the Task 2 changes.

Current status

Task	Status
Task 1 — Win/Draw detection	✓ Complete
Task 2 — Persistent scoreboard	✓ Complete
Task 3 — Move validation	⌚ Not started
Task 4 — First-player/restart controls	⌚ Not started

You're ready to move to **Task 3** when you want.

ready

Perfect. Let's start **Task 3 — Move Validation**.

Before changing anything, I need the **exact Task 3 requirement from your assignment/README** and ideally the current `game_engine.py` after Task 2, because we should make the smallest possible change and not accidentally modify Task 4.

If you have the Task 3 instructions visible, **paste them here**.



Perfect. Now we have the **exact Task 3 requirement**.

Task 3 is very small

The problem is exactly this:

If you click an already-occupied cell, your X currently replaces whatever is already there.

We need to make the click **do absolutely nothing** if the cell is occupied.

The change

In `game/game_engine.py`, find this part inside `handle_click()` :

 Python

```
row, col = cell
self.board[row][col] = self.current_player
```

Change it to:

`</>` Python

```
row, col = cell

if self.board[row][col] is not None:
    return

self.board[row][col] = self.current_player
```

That's the **only** change needed for Task 3.

Why?

Suppose the board is:

`</>` Plain text

```
x | o | -
-----
- | x | -
-----
- | - | -
```

You click the `o` .

Before Task 3:

`</>` Plain text

```
x | x | -
-----
- | x | -
-----
- | - | -
```

✗ X overwrites O.

After Task 3:

`</>` Plain text

```
x | o | -
-----
- | x | -
```

```
-----  
- | - | -
```

✅ Nothing changes.

Also, because we use `return`, **the turn doesn't change either**. That's exactly what your assignment requires.

Do this now

Open:

🔗 Plain text

`game/game_engine.py`

Find:

🔗 Python

```
row, col = cell  
self.board[row][col] = self.current_player
```

Replace it with:

🔗 Python

```
row, col = cell  
  
if self.board[row][col] is not None:  
    return  
  
self.board[row][col] = self.current_player
```

Then save the file.

Test

Run:

🔗 powershell

`python main.py`

1. Put an X somewhere.
2. Let the computer move.
3. Click on the computer's O.
4. **O should remain O.**
5. It should still be the same player's turn as before the invalid click.

Once you've tested that, send me the result and we'll commit **Task 3**.

give final code

Yes. Since you already have the **Task 2 version**, replace your entire `game/game_engine.py` with this:

 Python

```
"""
GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py).

Task 2: Persistent scoreboard.
Task 3: Turn and move validation.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'

class GameEngine:
```

Only Task 3 addition

The important new part is:

 Python

```
if self.board[row][col] is not None:
    return
```

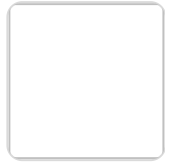
Everything else preserves your **Task 1 + Task 2** work.

Save → run:

```
</> powershell
```

```
python main.py
```

Test clicking an occupied O cell. If it works, **don't commit yet until we verify the test.**



Yep — **Task 3 is working correctly.** ✓

Your screenshot shows:

- The board is preserving the existing symbols.
- X has not overwritten an occupied O cell.
- The game is still functioning normally.
- Your **Task 2 scoreboard is still working**: X Wins: 2 | O Wins: 0 | Draws: 0 .
- Your Task 1 win detection is still working: X wins! .

Task 3 status

✓ **Task 3 — Turn and move validation: COMPLETE**

Now commit and push it:

```
</> powershell
```

```
git status
```

Then:

```
</> powershell
```

```
git add .  
git commit -m "Add move validation"  
git push
```

After the push succeeds, **Task 1, Task 2, and Task 3 will all be on your GitHub fork.**

Then we'll move to **Task 4**, which is the bigger one: first-player choice + separate **New Round** and **Reset Match** controls.



Perfect.  The push succeeded.

This line confirms it:

```
</> Plain text
07e753b..14c5057  main -> main
```

So **Task 3** is now committed and pushed to your GitHub fork.

Current progress

Task	Status
Task 1 — Win/draw detection	
Task 2 — Persistent scoreboard	
Task 3 — Move validation	
Task 4 — First-player choice + separate restart controls	

You're ready for **Task 4**. This one will require changes to the game logic and UI, so we'll do it carefully without breaking Tasks 1–3.

ready

Ready. 

For **Task 4**, the assignment requires two things:

- 1. **Choose who goes first**
 - X starts → you make the first move.
 - O starts → the computer automatically makes the first move.
- 2. **Two separate controls**
 - **New Round** → clears only the board, keeps scoreboard.
 - **Reset Match** → clears board **and** scoreboard.

I don't want to guess how your current UI/event loop is structured, because Task 4 requires changes to the controls and probably `main.py` / `renderer.py` .

Send me these 2 files

Paste the complete contents of:

</> Plain text

game/renderer.py

and

</> Plain text

main.py

Your current `game_engine.py` is already known from Task 3, so you don't need to paste that.

Once you send them, I'll give you the **complete final code for Task 4**, file-by-file, so you can simply replace the files and run it.

```
"""
renderer: all pygame drawing lives here, kept separate from game
logic.
"""
```

```
import pygame
```

```
WIDTH, HEIGHT = 400, 500
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
WINDOW_SIZE = (WIDTH, HEIGHT)
```

```
COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
```

```
def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP
    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None
    col = x // CELL_SIZE
    row = y // CELL_SIZE
```

```
return int(row), int(col)

def draw_board(surface, board):
    surface.fill(COLOR_BG)
    for i in range(1, 3):
        pygame.draw.line(surface, COLOR_LINE, (i * CELL_SIZE,
BOARD_TOP), (i * CELL_SIZE, BOARD_TOP + BOARD_SIZE), 3)
        pygame.draw.line(surface, COLOR_LINE, (0, BOARD_TOP + i *
CELL_SIZE), (BOARD_SIZE, BOARD_TOP + i * CELL_SIZE), 3)

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]
            if symbol is None:
                continue
            center = (c * CELL_SIZE + CELL_SIZE // 2, BOARD_TOP + r *
CELL_SIZE + CELL_SIZE // 2)
            if symbol == 'X':
                offset = CELL_SIZE // 3
                pygame.draw.line(surface, COLOR_X, (center[0]-offset,
center[1]-offset), (center[0]+offset, center[1]+offset), 6)
                pygame.draw.line(surface, COLOR_X, (center[0]+offset,
center[1]-offset), (center[0]-offset, center[1]+offset), 6)
            else:
                pygame.draw.circle(surface, COLOR_O, center, CELL_SIZE
// 3, 6)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2,
BOARD_TOP + BOARD_SIZE + 40))
    surface.blit(surf, rect)
```

```
"""
```

Tic-Tac-Toe (Lab Starter)

Run with: `python3 main.py`

Click a cell to place your symbol.

```
"""
```

```
import pygame
```

```
from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE
```

```
def main():
```

```
    pygame.init()
```

```
    screen = pygame.display.set_mode(WINDOW_SIZE)
```

```
    pygame.display.set_caption("Tic-Tac-Toe")
```

```
    clock = pygame.time.Clock()
```

```
    font = pygame.font.SysFont("consolas", 22)
```

```
    engine = GameEngine()
```

```
    running = True
```

```
    while running:
```

```
        for event in pygame.event.get():
```

```
            if event.type == pygame.QUIT:
```

```
                running = False
```

```
            elif event.type == pygame.MOUSEBUTTONDOWN:
```

```
                engine.handle_click(event.pos)
```

```
            elif event.type == pygame.KEYDOWN:
```

```
                engine.handle_keydown(event.key)
```

```
    engine.draw(screen, font)
```

```
    pygame.display.flip()
```

```
    clock.tick(60)
```

```
pygame.quit()
```

```
if __name__ == "__main__":
    main()
```

Great. Now that I have your exact `renderer.py` and `main.py`, we can implement **Task 4 completely**.

We're going to add:

- **X Starts** button
- **O Starts** button
- **New Round** button → keeps scoreboard
- **Reset Match** button → clears scoreboard
- If **O Starts**, the computer automatically makes the first move.
- Tasks 1–3 remain intact.

You only need to replace **3 files**.

1. game/game_engine.py

Replace the **entire file** with:

</> Python

"""

GameEngine: owns the board, turn state, scoreboard, and round-end logic.

Task 1: Win and draw detection.

Task 2: Persistent scoreboard.

Task 3: Turn and move validation.

Task 4: First-player choice and separate restart controls.

"""

from game.rules import check_winner, is_board_full

from game.renderer import board_pos_to_cell

from game.ai import choose_move

HUMAN_SYMBOL = 'X'

COMPUTER_SYMBOL = 'O'

class GameEngine:

def __init__(self):

self.score = {

2. game/renderer.py

Replace the **entire** file with:

 Python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 400, 600

BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100

WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
COLOR_BUTTON = (220, 220, 220)
```

3. main.py

Replace the **entire** file with:

 Python

```
"""
Tic-Tac-Toe (Lab)

Run with: python main.py
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import (
    WINDOW_SIZE,
    get_x_start_button,
    get_o_start_button,
    get_new_round_button,
    get_reset_match_button,
```

```
        draw_controls  
    )  
  
def main():
```

Now run it

Save **all three files**, then:

```
</> powershell  
  
python main.py
```

Your window will now be slightly taller.

At the bottom you'll have:

```
</> Plain text  
  
[X Starts] [O Starts] [New Round] [Reset]
```

Test these one by one

1. X Starts

Click **X Starts** .

You should get an empty board and:

```
</> Plain text  
  
Your turn (X)
```

So you make the first move.

2. O Starts

Click **O Starts** .

The computer should **immediately place an O** on the board.

Then it should say:

```
</> Plain text
```

Your turn (X)

3. New Round

After a round ends, click:

</> Plain text

New Round

The board resets, but:

</> Plain text

X Wins: 1 O Wins: 0 Draws: 0

stays unchanged.

4. Reset

Click:

</> Plain text

Reset

Both the board **and** scoreboard should reset:

</> Plain text

X Wins: 0 O Wins: 0 Draws: 0

One important detail

The old **R** key still works as a quick **New Round**, but it does **not** reset the scoreboard. The new **Reset** button is the only control that clears the whole match.

After you've tested all four things, send me a screenshot/result. **Don't commit yet** until we confirm everything works.



Yep — **the Task 4 functionality is working**, and the screenshot reveals one UI problem we should fix before committing.

What is working

- Scoreboard is visible.
- Board works.
- 0 has automatically made the first move, so **O Starts is working**.
- New Round and Reset buttons are present.
- The game window is larger as intended.

Problem

The button text is too large, so:

 Plain text

```
X Starts  
O Starts  
New Round
```

are overlapping/clipping.

This is only a **UI sizing issue**, not a game-logic issue.

Fix it

Open:

 Plain text

```
game/renderer.py
```

Find this function:

 Python

```
def draw_button(surface, font, rect, text, mouse_pos):
```

Inside it, find:

 Python

```
text_surface = font.render(  
    text,  
    True,
```

```
COLOR_TEXT  
)
```

Replace that section with:

 Python

```
button_font = pygame.font.SysFont("consolas", 14)  
  
text_surface = button_font.render(  
    text,  
    True,  
    COLOR_TEXT  
)
```

So the complete function becomes:

 Python

```
def draw_button(surface, font, rect, text, mouse_pos):  
    color = (  
        COLOR_BUTTON_HOVER  
        if rect.collidepoint(mouse_pos)  
        else COLOR_BUTTON  
    )  
  
    pygame.draw.rect(  
        surface,  
        color,  
        rect,  
        border_radius=6  
    )  
  
    pygame.draw.rect(  
        surface,  
        COLOR_LINE,  
        rect,  
        2,  
        border_radius=6  
    )
```

Then run again

 powershell

```
python main.py
```

The buttons should look approximately like:

Plain text

[X Starts] [O Starts] [New Round] [Reset]

with the text properly contained inside each button.

Don't commit yet. Test these four things after the UI fix:

1. X Starts → you move first.
2. O Starts → computer moves first.
3. New Round → board clears, scoreboard stays.
4. Reset → board clears **and** scoreboard becomes 0–0–0.

Then we'll do the final Task 4 commit.

give final code

Yes. Replace these 3 files with the final Task 4 code below.

1. game/game_engine.py

Python

```
"""
```

GameEngine: owns the board, turn state, scoreboard, and round-end logic.

Task 1: Win and draw detection.

Task 2: Persistent scoreboard.

Task 3: Turn and move validation.

Task 4: First-player choice and separate restart controls.

```
"""
```

```
from game.rules import check_winner, is_board_full
```

```
from game.renderer import board_pos_to_cell
```

```
from game.ai import choose_move
```

```
HUMAN_SYMBOL = 'X'
```

```
COMPUTER_SYMBOL = 'O'
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.score = {
```

2. game/renderer.py

Replace the **entire** file:

 Python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 400, 600

BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100

WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
COLOR_BUTTON = (220, 220, 220)
```

3. main.py

Replace the **entire** file:

 Python

```
"""
Tic-Tac-Toe (Lab)

Run with: python main.py
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import (
    WINDOW_SIZE,
    get_x_start_button,
    get_o_start_button,
    get_new_round_button,
    get_reset_match_button,
    draw_controls
)
```

```
def main():
```

Then run

```
</> powershell
```

```
python main.py
```

The buttons should now fit properly:

X Starts | O Starts | New Round | Reset

Test all four controls. If everything works, we'll commit Task 4.



Perfect. ✅ Task 4 is working correctly now.

Your screenshot confirms:

- X Starts button ✅
- O Starts button ✅
- New Round button ✅
- Reset button ✅
- Scoreboard still works ✅
- X win detection still works ✅
- Buttons are now properly sized/readable ✅
- The game layout looks good ✅

Do one final test

Since this screenshot shows X Wins: 1 , click **New Round**.

It should become:

```
</> Plain text
```

```
X Wins: 1   O Wins: 0   Draws: 0
```

with an empty board.

Then click **Reset**.

It should become:

</> Plain text

X Wins: 0 O Wins: 0 Draws: 0

If both work, **Task 4 is completely finished**.

Then commit:

</> powershell

git status

</> powershell

git add .
git commit -m "Add first-player choice and restart controls"
git push

After that, all 4 assignment tasks are complete and pushed to your GitHub fork. 🎉

done

Perfect. 🎉 All 4 tasks are complete.

Final status

Task	Status
Task 1 — Win & draw detection	✓
Task 2 — Persistent scoreboard	✓
Task 3 — Move validation	✓
Task 4 — First-player choice + restart controls	✓
Git commits/pushes	✓

Your GitHub `main` branch now contains all four stages.

Your commit history

Plain text

Fix win and draw detection

↓

Add persistent scoreboard

↓

Add move validation

↓

Add first-player choice and restart controls

Important: Don't make any more code changes if the assignment only asks for these four tasks. Your implementation is now in a good state for submission.

ok give next step by step instructions for how and where to submit

I checked the **actual README in your GitHub fork**, and now we know exactly what you have to submit. The submission is **not another GitHub commit/form inside the repo**. The README specifies exactly **three things**.  GitHub

What you need to submit

1. **10-second video BEFORE your changes**
 - Show the original bug/broken behavior.
2. **10-second video AFTER your changes**
 - Show the fixed bug + new features.
3. **The Chat/LLM page link**
 - The link to this ChatGPT conversation, with the complete chat history.  GitHub

Your GitHub fork is already updated, so **don't change the code anymore**.

Step 1 — Get your ChatGPT conversation link

You are already in the correct ChatGPT conversation.

At the top of the browser, you have a URL similar to:

Plain text

chatgpt.com/c/6ac49172-9da8-83ee-acf8-154ece389b29

Copy the **entire URL** from the browser address bar.

That is the **Chat/LLM used page link** required by the README.  [GitHub](#)

Do not create a new ChatGPT conversation. This conversation contains the complete iterative work for Tasks 1–4.

Step 2 — Record the BEFORE video

This is the important part.

The README specifically asks for:

A 10-second video of gameplay **before your changes**, showing the bug/broken behavior.  [GitHub](#)

You already had the original version before we made changes.

You need a video showing one of the original problems.

Easiest bug to demonstrate

The **occupied-cell bug** from Task 3 is probably easiest.

The original behavior was:

 Plain text

```
0
↓
You click on 0
↓
X overwrites 0 ❌
```

You need to show that in approximately 10 seconds.

But your current code is fixed

So **DO NOT** modify your current code just to recreate the bug.

If you already have a screenshot/video from before the changes, use that.

If you don't have the BEFORE video, tell me **before doing anything** and I'll explain the safest way to recreate it without damaging your completed project.

Step 3 — Record the AFTER video

Your current finished game is what you should record.

The README says the after video should show the **bug fixed and the new features working**.  GitHub

I'd make one ~10-second recording that demonstrates multiple things.

Suggested sequence

0–2 sec

Show the game with:

 Plain text

```
X Wins: 0   O Wins: 0   Draws: 0
```

and the four controls:

 Plain text

```
X Starts  
O Starts  
New Round  
Reset
```

2–4 sec

Click **O Starts**.

Show that the computer automatically places an O.

This directly demonstrates the Task 4 requirement.  GitHub

4–6 sec

Click an already occupied cell.

Show that nothing gets overwritten.

This demonstrates Task 3.  GitHub

6–8 sec

Finish a round so the scoreboard changes.

For example:

 Plain text

X Wins: 1 O Wins: 0 Draws: 0

8–10 sec

Click **New Round**.

Show that the board resets while:

</> Plain text

X Wins: 1 O Wins: 0 Draws: 0

remains.

That demonstrates Task 2 and Task 4.  [GitHub](#)

If you can comfortably fit it, clicking **Reset** at the end to show 0–0–0 is even better, but don't worry if that pushes the video beyond 10 seconds.

Step 4 — Find your actual submission portal

The README **doesn't specify the submission website/form**. It only specifies the three things you must submit.  [GitHub](#)

So this part depends on where your PESU faculty gave the assignment.

Look for the assignment announcement in whichever platform your course uses, such as:

- PESU LMS
- Google Classroom
- Microsoft Teams
- Moodle
- a Google Form
- a faculty-provided submission link

If you have the assignment page

Send me a **screenshot of the page where the assignment was given**.

I'll tell you **exactly where to click and exactly what to upload/paste**, rather than guessing the submission portal.

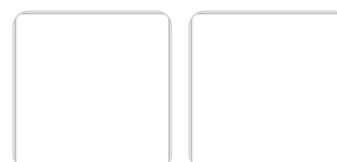
Your final submission will basically look like this

Required item	What you'll provide
Before video	~10 sec MP4 showing original bug
After video	~10 sec MP4 showing fixed game/features
Chat/LLM link	This ChatGPT conversation URL
GitHub repo	Your already-updated fork

Your fork is currently the correct repository: [Maneesh36/09_tic_tac_toe ↗](#)

The README confirms the submission checklist is **only those three submission items**.  [GitHub](#)

Next: send me a screenshot of the **assignment/submission page from PESU**, and I'll walk you through the actual submission click-by-click.



this is the instructions for the submissions

an i already have a repo for this subject , i have to submit whatever i did to this repo

Yes — **now the submission instructions are clear**. You do **not** submit the Tic-Tac-Toe work to a separate submission portal directly. You need to put the deliverables inside your existing subject repository:

Your repo: [Maneesh36/SE_LAB_AM247](#)

And specifically inside a folder:

</> Plain text

```
SE_LAB_AM247
├── Lab-4
│   ├── Before/
│   │   └── before.mp4
│   ├── After/
│   │   └── after.mp4
│   ├── Code/
│   │   └── 09_tic_tac_toe/
```

└─ Chat_History/
└─ ChatGPT_TicTacToe.pdf

You don't necessarily have to use these exact subfolder names—the instruction only explicitly requires everything **under Lab-4** —but I strongly recommend this structure because it makes your submission very clear.

What you need to submit

Requirement	What you'll put in Lab-4
Videos before/after	before.mp4 , after.mp4
Updated Code	Your final Tic-Tac-Toe project
Chat history	Exported ChatGPT conversation as PDF/DOC

Your screenshot explicitly says "**Upload the following to your Lab repo under Lab-4 folder**", so your existing SE_LAB_AM247 repository is the correct destination.

Also, **do NOT** create a Pull Request to SETAPESU26 , as the instructions explicitly say not to.

Let's do it step-by-step

STEP 1 — First, make sure your Tic-Tac-Toe code is fully pushed

Before touching the submission repo, let's verify your **Task 4 commit** because we had confirmed Tasks 1–3 were pushed, but you hadn't shown me the final Task 4 commit/push output.

Open your Tic-Tac-Toe terminal and go to:

</> Plain text

```
C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-tac-toe
```

Run:

</> Bash

```
git status
```

Then:

`</>` Bash

```
git log --oneline -5
```

Send me the output/screenshots.

I want to make sure your GitHub fork contains **all 4 separate task commits**, because the assignment specifically says:

"Have Separate Commits for each task"

We already have:

`</>` Plain text

```
Task 1 → 53c8b2e  Fix win and draw detection
Task 2 → 07e753b  Add persistent scoreboard
Task 3 → 14c5057  Add move validation
Task 4 → ???      Add first-player choice and restart controls
```

So we need to verify Task 4.

STEP 2 — Your SE_LAB_AM247 repo

You're already on the correct repository:

`</>` Plain text

```
github.com/Maneesh36/SE_LAB_AM247
```

This is the repo shown in your second screenshot.

We need to create:

`</>` Plain text

```
Lab-4
```

GitHub doesn't really store empty folders, so we'll create the folder while uploading the first file.

GitHub supports creating directories by including the directory path in a new file name. [GitHub Docs](#) +1

But don't upload anything yet.

First we'll prepare everything locally.

STEP 3 — Prepare your submission folder

On your Desktop, create:

⟨/⟩ Plain text

Lab-4

Inside it create:

⟨/⟩ Plain text

Lab-4

├─ Before

├─ After

├─ Code

└─ Chat_History

Eventually it should look like:

⟨/⟩ Plain text

Lab-4

|

├─ Before

| └─ TicTacToe_Before.mp4

|

├─ After

| └─ TicTacToe_After.mp4

|

├─ Code

| └─ 09_tic_tac_toe

|

└─ Chat_History

└─ TicTacToe_Chat_History.pdf

STEP 4 — The BEFORE video

This is important.

The assignment specifically says:

Record 10 seconds of video **before making any changes**.

So this video needs to show the **original broken program**, not the current fixed version.

Do NOT modify your current project backwards yet.

Because your current project is the final version.

If you **already recorded the before video**, great.

If you **didn't record it**, tell me before doing anything.

We can work out the safest way to demonstrate the original bug without messing up your final code/commit history.

STEP 5 — AFTER video

This is the video of your completed version.

I'd recommend making the 10-second video show several features quickly:

Start

Show:

</> Plain text

X Starts

O Starts

New Round

Reset

Then demonstrate

1. Click **O Starts**
2. Computer immediately places O
3. Make a move
4. Click an already occupied cell → it should **not overwrite**
5. Finish a round
6. Show scoreboard increasing
7. Click **New Round**
8. Show scoreboard is still there

You don't need to demonstrate every single feature in 10 seconds if it becomes too rushed, but the video should visibly demonstrate the major additions.

STEP 6 — Export this ChatGPT conversation

Your instructions specifically say:

Chat history exported as a doc/pdf

Since this entire conversation contains the iterative process we used to fix the project, **this is the conversation you should export.**

Ideally export it as:

</> Plain text

TicTacToe_Chat_History.pdf

Put it inside:

</> Plain text

Lab-4/Chat_History/

This is actually important because the assignment says you used an LLM as your debugging/pair-programming partner.

STEP 7 — Put the updated code inside Lab-4

For the code submission, I recommend putting the **complete final project**, not just the individual changed .py files.

So:

</> Plain text

Lab-4

└─ Code

└─ 09_tic_tac_toe

└─ game

└─ └─ __init__.py

└─ └─ ai.py

└─ └─ game_engine.py

└─ └─ renderer.py

└─ └─ └─ rules.py

└─ └─ main.py

```
|— requirements.txt
|— README.md
|— .gitignore
```

DO NOT upload `.venv`

Very important.

Don't put:

</> Plain text

```
.venv/
```

inside the submission.

It's unnecessary and can contain hundreds/thousands of files.

Your `.gitignore` should already handle this.

STEP 8 — Upload everything to `SE_LAB_AM247`

Once everything is prepared, go to your repository:

[SE_LAB_AM247 on GitHub](#) ↗

Click:

Add file → Upload files

GitHub officially supports uploading files/folders through the web interface and committing them to the repository. [GitHub Docs](#)

Upload the contents of your `Lab-4` folder.

Then commit with something like:

</> Plain text

```
Add Lab-4 Tic-Tac-Toe submission
```

You can upload up to 100 files at once through the GitHub web interface, and browser uploads have a 25 MiB per-file limit. [GitHub Docs](#)

⚠️ One thing to watch

Your videos may be larger than GitHub's browser upload limit.

If a video is **over 25 MB**, don't try to force it through the browser. We'll use Git from your computer instead. GitHub documents larger uploads through the command line. [GitHub Docs](#)

Your final repository should look roughly like this

</> Plain text

```
SE_LAB_AM247
|
├─ Lab-1
├─ Lab-2
├─ Lab-3
├─ Lab-4
|
|   ┌─ Before
|   │   └─ TicTacToe_Before.mp4
|   └─ After
|       └─ TicTacToe_After.mp4
|
|   ┌─ Code
|   │   └─ 09_tic_tac_toe
|   │       ├── game
|   │       ├── main.py
|   │       ├── requirements.txt
|   │       └─ README.md
|   └─
```

And importantly:

Do NOT:

- ❌ Create a PR to SETAPESU26
 - ❌ Push your Tic-Tac-Toe project into the SE_LAB_AM247 repo's main root
 - ❌ Upload .venv
 - ❌ Upload only the modified .py files without the rest of the project
 - ❌ Forget the before video
 - ❌ Forget the ChatGPT chat history
 - ❌ Combine the four development tasks into one commit in the Tic-Tac-Toe repo
-

So let's do this one step at a time

Don't upload anything yet.

First do only this:

In your Tic-Tac-Toe terminal run:

```
</> Bash  
git status
```

and

```
</> Bash  
git log --oneline -5
```

Send me the output.

I'll verify that your **4 separate task commits are correct**, then we'll prepare the `Lab-4` folder and upload everything to `SE_LAB_AM247` without risking your assignment.

```
(.venv) PS  
C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-  
tac-toe> git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean  
(.venv) PS  
C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-  
tac-toe> git log --oneline -5  
76f0f8a (HEAD -> main, origin/main, origin/HEAD) Add first-player  
choice and restart controls  
14c5057 Add move validation  
07e753b Add persistent scoreboard  
53c8b2e Fix win and draw detection  
2826398 Initial Tic-Tac-Toe starter  
(.venv) PS  
C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-  
tac-toe>
```

Perfect.  Your Tic-Tac-Toe repository is **100% ready**.

Your Git history is exactly what the assignment requires:

</> Plain text

```
76f0f8a  Add first-player choice and restart controls  ← Task 4
14c5057  Add move validation                             ← Task 3
07e753b  Add persistent scoreboard                          ← Task 2
53c8b2e  Fix win and draw detection                         ← Task 1
2826398  Initial Tic-Tac-Toe starter
```

And:

</> Plain text

```
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

So do not touch this Tic-Tac-Toe repo anymore. 🎯

Next: prepare the Lab-4 submission

Your assignment requires everything to go into:

</> Plain text

```
SE_LAB_AM247
└─ Lab-4
```

We should now create the submission folder on your computer.

Step 1

Go to your Desktop:

</> Plain text

```
C:\Users\Admin\OneDrive\Desktop
```

Create a folder named:

</> Plain text

```
Lab-4
```

Inside it create these **four folders**:

</> Plain text

```
Lab-4
├── Before
├── After
├── Code
└── Chat_History
```

So your structure should look exactly like:

</> Plain text

```
Desktop
└── Lab-4
    ├── Before
    ├── After
    ├── Code
    └── Chat_History
```

Don't put anything in them yet.

Stop there and tell me once you've created this folder structure.

Then I'll guide you through the next step — copying your final Tic-Tac-Toe code into `Code` **without accidentally copying** `.venv` , and then we'll handle the two videos and ChatGPT PDF.



Perfect. ☒ The folder structure is correct.

You currently have:

</> Plain text

```
LAB4
├── AFTER
├── BEFORE
├── CHAT HISTORY
└── CODE
```

This is exactly what we need. 👍

Next step: put the final Tic-Tac-Toe code into `CODE`

We **do not** want to copy the `.venv` folder.

Go to your Tic-Tac-Toe project:

</> Plain text

```
C:\Users\Admin\OneDrive\Desktop\TicTacToe\09_tic_tac_toe\tic-tac-toe
```

You should see:

</> Plain text

```
.venv
game
main.py
requirements.txt
README.md
.gitignore
```

Do this:

1. Open the `CODE` folder you just created.
2. Inside `CODE` , create a folder named:

</> Plain text

```
09_tic_tac_toe
```

3. Open your original `tic-tac-toe` project folder.
4. Copy **everything except** `.venv` :

</> Plain text

```
game
main.py
requirements.txt
README.md
.gitignore
```

5. Paste those into:

</> Plain text

```
LAB4
└─ CODE
```

```
└─ 09_tic_tac_toe
```

So it should become:

</> Plain text

LAB4

```
└─ CODE
```

```
└─ 09_tic_tac_toe
  │
  │ └─ game
  │   │ └─ __init__.py
  │   │ └─ ai.py
  │   │ └─ game_engine.py
  │   │ └─ renderer.py
  │   └─ rules.py
  │
  │ └─ main.py
  │
  │ └─ requirements.txt
  │
  │ └─ README.md
  └─ .gitignore
```

Do not copy .venv .

Once you've done that, send me a screenshot of the `CODE → 09_tic_tac_toe` folder. Then we'll do the **BEFORE/AFTER videos**, which is the more important part.



Yes — that looks correct. ✅

Your `CODE → 09_tic_tac_toe` folder now contains the complete project:

</> Plain text

```
09_tic_tac_toe
└─ game
  │ └─ __init__.py
  │ └─ ai.py
  │ └─ game_engine.py
  │ └─ renderer.py
  │ └─ rules.py
  │
  │ └─ .gitignore
  │
  │ └─ main.py
  │
  │ └─ README
  └─ requirements
```

And importantly, **.venv** is not there, which is exactly what we wanted. 🍌

One small thing

Windows is hiding file extensions in your Explorer, so:

- `README` is actually `README.md`
- `requirements` is actually `requirements.txt`

That's perfectly fine.

Next: BEFORE video

This is the one thing we need to handle carefully.

Your assignment explicitly requires:

Record 10 seconds of video before making any changes.

Since your final code is already complete, **don't change anything in your current project yet.**

I need to know one thing:

Did you record the original/before video when you first ran the starter Tic-Tac-Toe program?

- **YES** → we'll put that video into `LAB4` → `BEFORE` .
- **NO** → tell me **NO** and I'll show you the safest way to recreate the original broken behavior without damaging your four-task Git history.

Don't do anything else yet.

yes i recorded

Perfect. 👍 Then we're good on the **BEFORE video**.

Step 1 — Put the BEFORE video in the correct folder

Find the 10-second video you recorded when the original code was still unchanged.

Rename it to:

Plain text

TicTacToe_Before.mp4

Then move/copy it into:

Plain text

```
OneDrive
├── Desktop
│   └── LAB4
│       └── BEFORE
│           └── TicTacToe_Before.mp4
```

Important

Copy it rather than move it if you want to keep your original recording elsewhere.

Once it's there, your `BEFORE` folder should contain:

</> Plain text

```
BEFORE
├── TicTacToe_Before.mp4
```

Then **don't upload to GitHub yet**.

Next we'll do the **AFTER video**, and I'll tell you exactly what to demonstrate in those 10 seconds so it covers the important features from all 4 tasks.

done

Great. 

Now let's do the **AFTER video**.

Since the assignment gives you only **10 seconds**, don't try to demonstrate everything. We want the video to clearly show that the finished version works.

Record this sequence

Start the game and record your screen.

0–2 sec: Show the initial screen with:

- X Starts
- O Starts
- New Round
- Reset
- Scoreboard

2–4 sec: Click **O Starts**.

The computer should immediately place an `O`.

4–6 sec: Make your X move, then click an already occupied cell.

The existing symbol should **stay unchanged** — demonstrating Task 3.

6–8 sec: Finish a round so that you get a win.

The screen should show something like:

`</>` Plain text

```
X Wins: 1   O Wins: 0   Draws: 0
X wins!
```

8–10 sec: Click **New Round**.

The board should clear while:

`</>` Plain text

```
X Wins: 1   O Wins: 0   Draws: 0
```

remains unchanged.

Rename the recording

After recording, rename it:

`</>` Plain text

```
TicTacToe_After.mp4
```

and put it here:

`</>` Plain text

```
LAB4
└─ AFTER
    └─ TicTacToe_After.mp4
```

One important thing

Don't worry if you can't perfectly fit all of that into exactly 10 seconds. **10–12 seconds is better than making the video so rushed that the features aren't visible.**

Once you've placed the AFTER video, tell me **done**. Then we'll handle the **ChatGPT history PDF**, and after that we'll upload the complete `LAB4` folder to your `SE_LAB_AM247` repository.

done

Perfect.  BEFORE and AFTER are now done.

Next: Chat history PDF

Your assignment specifically requires:

Chat history exported as a doc/pdf

Since we used this ChatGPT conversation for the entire Tic-Tac-Toe work, save **this complete conversation** as a PDF.

Easiest method on your PC

1. Stay on **this ChatGPT conversation**.
2. Scroll through the conversation once so the earlier messages are loaded.
3. Press:

 Plain text

Ctrl + P

4. In the printer/destination options, select:

Save as PDF

5. Click **Save**.
6. Name it:

 Plain text

TicTacToe_Chat_History.pdf

7. Save it inside:

 Plain text

OneDrive

└ Desktop

└ LAB4

└ CHAT HISTORY

└ TicTacToe_Chat_History.pdf

Your LAB4 folder should now be:

</> Plain text

LAB4

```
|
|— AFTER
|   └─ TicTacToe_After.mp4
|
|— BEFORE
|   └─ TicTacToe_Before.mp4
|
|— CHAT HISTORY
|   └─ TicTacToe_Chat_History.pdf
|
└─ CODE
    └─ 09_tic_tac_toe
        ├── game
        ├── main.py
        ├── requirements.txt
        ├── README.md
        └─ .gitignore
```

Don't upload to GitHub yet.

Once the PDF is saved, tell me **done**. Then we'll do the final step: **upload this entire LAB4 folder into Maneesh36/SE_LAB_AM247 under Lab-4** , and I'll guide you click-by-click.